

1. Start.
2. Given a File of N employee records with a set K of Keys (4-digit) which uniquely determine the records in file F .
3. Assume that file F is maintained in memory by a Hash Table (HT) of m memory locations with L as the set of memory addresses (2-digit) of locations in HT.
4. Let the keys in K and addresses in L are Integers
5. Hash function $H: K \rightarrow L$ as $H(K) = K \bmod m$ (remainder method)
6. Hashing as to map a given key K to the address space L , Resolve the collision (if any) is using linear probing.
7. Stop.

```
#include <stdio.h>
#include <conio.h>
#define MAX 10
void create (int a[]) { int i;
for (i=0; i<MAX; i++) a[i] = -1;
printf("Hash table created \n"); }
int hashvalue (int value, int j)
{ int hash;
hash = ((value % MAX) + j) % MAX;
return (hash); }
```

```
void display (int a[]) { int i;
for (i=0; i<MAX; i++)
printf("%d → %d", i, a[i]); }
void insert (int a[], int value) {
int i, j, loc, flag = 0;
for (i=0; i<MAX; i++)
if (a[i] == -1) { flag = 1; break; }
if (flag == 0)
{ printf("Hash table full. Overflow
condition \n");
return; }
j = 0;
while (1) {
loc = hashvalue (value, j);
```

```
if (a[loc] == -1)
```

```
{ a[loc] = value;
```

```
printf("Value inserted \n");
```

```
display(a); return; }  
else j++; } }
```

```
void search (int a[], int value) {  
    int i, j, loc, flag = 0;
```

```
    for (i = 0; i < MAX; i++)
```

```
        if (a[i] == -1) flag = 1; break; }
```

```
    if (flag == 0) {
```

```
        printf("Hash table empty \n");
```

```
        return; }
```

```
    j = 0;
```

```
    while (j < MAX) {
```

```
        loc = hashvalue(value, j);
```

```
        if (a[loc] == value) {
```

```
            printf("Value found at position
```

```
            %d \n", loc); return; }
```

```
        else j++; }
```

```
    printf("Value not found \n"); }
```

```
void delete (int a[], int value) {
```

```
    int i, j, loc, flag = 0;
```

```
    for (i = 0; i < MAX; i++)
```

```
        if (a[i] == -1) { flag = 1; break; }
```

```
    if (flag == 0) {
```

```
        printf("Hash table empty \n"); return; }
```

```
    j = 0;
```

```
    while (j < MAX) {
```

```
        loc = hashvalue(value, j);
```

```
        if (a[loc] == value) { a[loc] = -1;
```

```
        printf("Value deleted \n");
```

```
        display(a); return; }
```

```
    else j++; }
```

```
    printf("Value not found \n"); }
```

```
void main () {
```

```
    int a[MAX], choice, key;
```

```
    clrscr(); create(a);
```

```
    while (1) {
```

```
        printf("\n MENU: 1. Create
```

```
        2. Insert 3. Delete 4. Search
```

```
        5. Exit ");
```

```
        printf("Enter your choice: ");
```

```
        scanf("%d", &choice);
```

```
        switch (choice) {
```

```
            case 1: create(a); break;
```

```
            case 2: printf("Enter value to be
```

```
            inserted: "); scanf("%d", &key);
```

```
            insert(a, key); break;
```

```
            case 3: printf("Enter value to
```

```
            be deleted"); scanf("%d", &key);
```

```
            delete(a, key); break;
```

```
            case 4: printf("Enter value to be
```

```
            searched"); scanf("%d", &key);
```

```
            search(a, key); break;
```

```
            case 5: exit(0);
```

```
        } }
```